

Probability Sampling

September 16, 2026

Applied Example: Using a Sample to Estimate Low Birthweight in U.S. Territories

Description of the Data

These data are births by place of occurrence for U.S. territories (American Samoa, Guam, North Mariana Islands, Puerto Rico, and U.S. Virgin Islands) from the year 2015.

This is a subset of the data downloaded from the National Bureau of Economic Research. You can find more information about the data set in the User Guide.

Data Dictionary

Here is the data dictionary for this dataset:

Variable	Description
babyID	Unique identifier: row number
dbwt	Birth weight in Grams: 227-8165 grams
combgest	Combined gestation, in weeks: 17th to 47th week of gestation
sex	Assigned sex at birth: M (Male) or F (Female)
dob_mm	Birth month
cig_rec	If the mother reports smoking in any of the three trimesters of pregnancy she is classified as a smoker: (Y) Yes, (N) No, or (U) Unknown

Import the Data into R

```
library(tidyverse)
birth_data <- read_csv(file = "./L03_US-territories-births.csv")
birth_data <- birth_data %>% select(-...1)
```

```
head(birth_data)
```

```
## # A tibble: 6 x 6
##   babyID dbwt combgest sex  dob_mm cig_rec
##   <dbl> <dbl>   <dbl> <chr>  <dbl> <chr>
## 1     1   2977     37 M      1 N
## 2     2   3191     41 M      1 Y
## 3     3   1786     32 F      1 N
## 4     4   4489     39 M      1 N
## 5     5   3203     38 M      1 N
## 6     6   3203     39 F      1 N
```

Overview of the Applied Example

- Take a **simple random sample** from the **population** of births. We will use this sample to *estimate* the **sample proportion** of babies with low birth weight in the population.
- To know how close our **estimate** is to the **true proportion**, we will first calculate the **population proportion** of an infant being born less than 5 pounds 8 ounces, or 2,500 grams, which is the traditional cutoff used to classify an infant as having low birth weight.
- In real life settings, we would not know the **true** proportion, we would only know the value we estimate from our sample. But here, we can investigate how close the sample estimate is to the true parameter and see what we can do to make our estimate even better.
- We will see how well we can estimate the true probability based on random samples of varying sizes.

Step 1: Add a Variable to the Dataset for Low Birth Weight (LBW)

```
birth_data <- birth_data %>% mutate(lbw = dbwt < 2500)
```

- What does the variable `lbw` store?
- `lbw` stores “logical” values, which means it is either equal to `TRUE` or `FALSE`
- Variables that store only two values are called **binary** variables. They are most commonly stored as logical data (`TRUE/FALSE`), numeric (`0/1`) or categorical (“Yes”/“No”).
- In the above code, R evaluates whether birth weight (`dbwt`) is less than 2,500 grams for each birth. If it is, then `lbw = TRUE` for that birth, and if not then `lbw = FALSE` for that birth.

Step 2: Calculate the Proportion of Low Birth Weight Infants in Population Across All U.S. Territories

```
lbw_population <- birth_data %>% summarize(true_prob_lbwt = mean(lbw))
lbw_population
```

```
## # A tibble: 1 x 1
##   true_prob_lbwt
##         <dbl>
## 1         0.102
```

```
mean(birth_data$lbwt)
```

```
## [1] 0.1023309
```

- Question: How did we take a mean of values equal to `TRUE` or `FALSE`?
- Answer: R treats `TRUE` as equivalent to 1 and `FALSE` as equivalent to 0. The mean of a variable coded as 0 or 1 is the **proportion** (or **probability**) of individuals who have low birth weight.
- Remember: We do not usually know the true value because we rarely have data on every individual in a population

Step 3: Take a Random Sample of Size $n=10$

```
random_sample_n10 <- birth_data %>%
  slice_sample(n = 10) %>%
  mutate(sample_size = n())
#the last line of code adds the sample size to every row of the new data frame
#we will want to reference this information later
```

Step 4: Progressively Increase the Sample Size and Store those Samples

Sample the rows of data using the following sample sizes. Assign each of your samples to a different R object.

1. 10
2. 50
3. 100
4. 200
5. 500
6. 1000
7. 5000
8. 10000
9. 36724 (i.e, the entire target population)

Step 4 Code

```
random_sample_n50 <- birth_data %>% slice_sample(n = 50) %>% mutate(sample_size = n())
random_sample_n100 <- birth_data %>% slice_sample(n = 100) %>% mutate(sample_size = n())
random_sample_n200 <- birth_data %>% slice_sample(n = 200) %>% mutate(sample_size = n())
random_sample_n500 <- birth_data %>% slice_sample(n = 500) %>% mutate(sample_size = n())
random_sample_n1000 <- birth_data %>% slice_sample(n = 1000) %>% mutate(sample_size = n())
random_sample_n5000 <- birth_data %>% slice_sample(n = 5000) %>% mutate(sample_size = n())
random_sample_n10000 <- birth_data %>% slice_sample(n = 10000) %>% mutate(sample_size = n())
whole_pop <- birth_data %>% slice_sample(n = nrow(birth_data)) %>% mutate(sample_size = n())
```

Step 4 Side Note

By default `slice_sample(n=100)` takes a sample of size 100 **without replacement**. This means that each individual can only be included in the sample at most one time.

In future classes, we will introduce methods where we select a sample **with replacement**. We will talk more about how this works in the coming weeks.

Step 5: Calculate the Estimate of the Proportion of LBW for Each Random Sample

- For each sample, we want to estimate the proportion of LBW babies to see how much it differs from the true proportion in the entire population

- We could do this using `summarize` for each sample and by writing that code ten times, but there is an easier way

Step 5: Code to Estimate the Proportions More Efficiently

The function `bind_rows(df1, df2, df3, ...)` can be used to stack multiple data frames (e.g. `df1`, `df2`, `df3`, ...) on top of each other when they all contain the same variables. Bind together the nine data frames created in the previous code chunk using `bind_rows()` and assign the stacked data frame the name `stacked_samples`:

```
stacked_samples <- bind_rows(random_sample_n10, random_sample_n50,
                             random_sample_n100, random_sample_n200,
                             random_sample_n500, random_sample_n1000,
                             random_sample_n5000, random_sample_n10000,
                             whole_pop)
```

Step 5: Code to Estimate the Proportions More Efficiently

Estimate the proportion of babies with low birth weight using each of your samples in `stacked_samples`. Hint: `group_by()` and `summarize()` will come in handy! Assign the output to a data frame called `sample_estimates`

```
sample_estimates <- stacked_samples %>%
  group_by(sample_size) %>%
  summarize(estimated_proportion_lbwt = mean(lbw))

sample_estimates
```

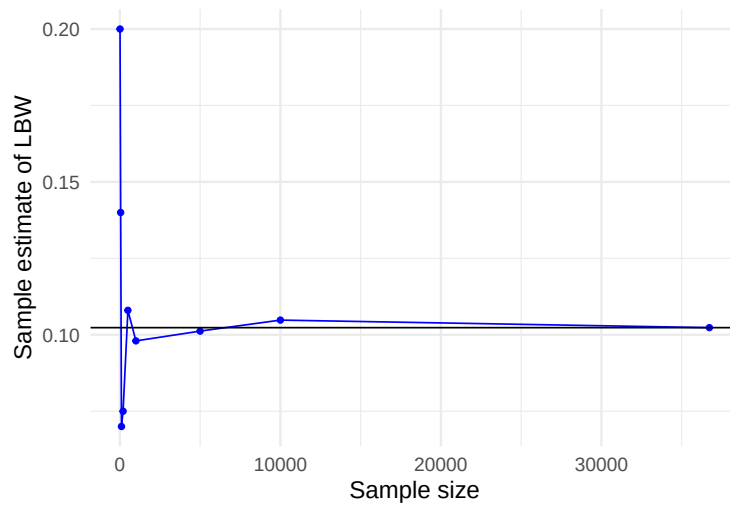
```
## # A tibble: 9 x 2
##   sample_size estimated_proportion_lbwt
##   <int>          <dbl>
## 1      10          0.2
## 2      50          0.14
## 3     100          0.07
## 4     200          0.075
## 5     500          0.108
## 6    1000          0.098
## 7    5000          0.101
## 8   10000          0.105
## 9   36724          0.102
```

Step 6: Visualize the Results

- Make a line plot of the estimates of the proportions versus the sample size
- Add a horizontal line to the line plot at the **population proportion** that we are striving to estimate
- We also add points on top of the line to see exactly where the estimates are

```
ggplot(sample_estimates, aes(x = sample_size, y = estimated_proportion_lbwt)) +
  geom_line(col = "blue") +
  geom_point(col = "blue") +
  geom_hline(yintercept = lbwt_population %>% pull(true_prob_lbwt)) +
```

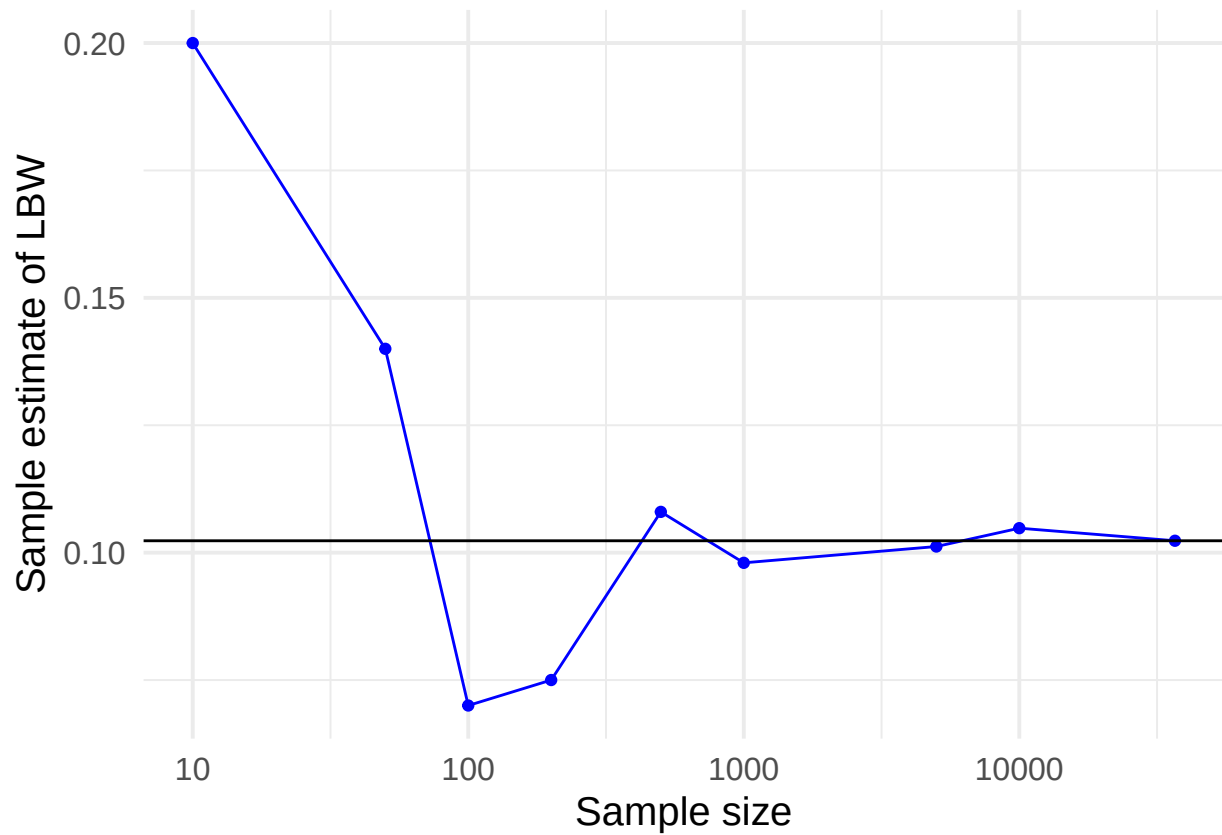
```
labs(y = "Sample estimate of LBW", x = "Sample size") +
theme_minimal(base_size = 15)
```



Step 6: Visualize the Results

- Because the scale of the x axis is so large, try using `scale_x_log10()` to convert the x-axis to a logarithmic scale

```
ggplot(sample_estimates, aes(x = sample_size, y = estimated_proportion_lbw)) +
  geom_line(col = "blue") +
  geom_point(col = "blue") +
  geom_hline(yintercept = lbw_population %>% pull(true_prob_lbw)) +
  scale_x_log10() +
  labs(y = "Sample estimate of LBW", x = "Sample size") +
  theme_minimal(base_size = 15)
```



Check Your Understanding!

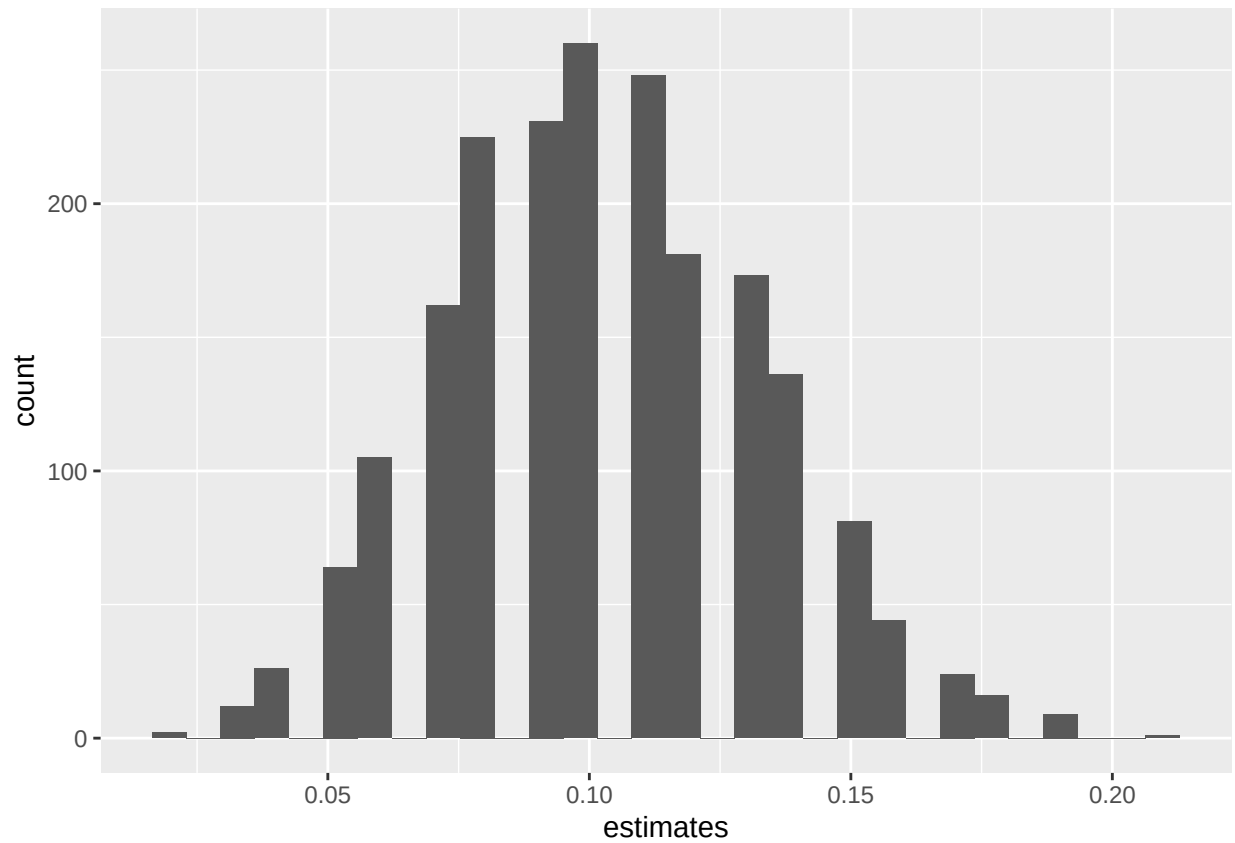
- 1) What happens as the sample size increases?
- 2) Will it **always** be the case that a higher sample size produces an estimate closer to the true value than a lower sample size?

sampling variability

```
nrep = 2000
estimates_n_100 = rep(NA, nrep)
for (i in 1:nrep){
  set.seed(i)
  random_sample_n100 <- birth_data %>% slice_sample(n = 100)
  estimates_n_100[i] <- mean(random_sample_n100$lbw)
}

histogram_100 <- ggplot(data = data.frame(estimates = estimates_n_100), aes(x=estimates)) + geom_histogram()
histogram_100
```

'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.



```
mean(estimates_n_100)
```

```
## [1] 0.102835
```

```
sd(estimates_n_100)
```

```
## [1] 0.03060525
```

```
mean(estimates_n_100)
```

```
## [1] 0.102835
```

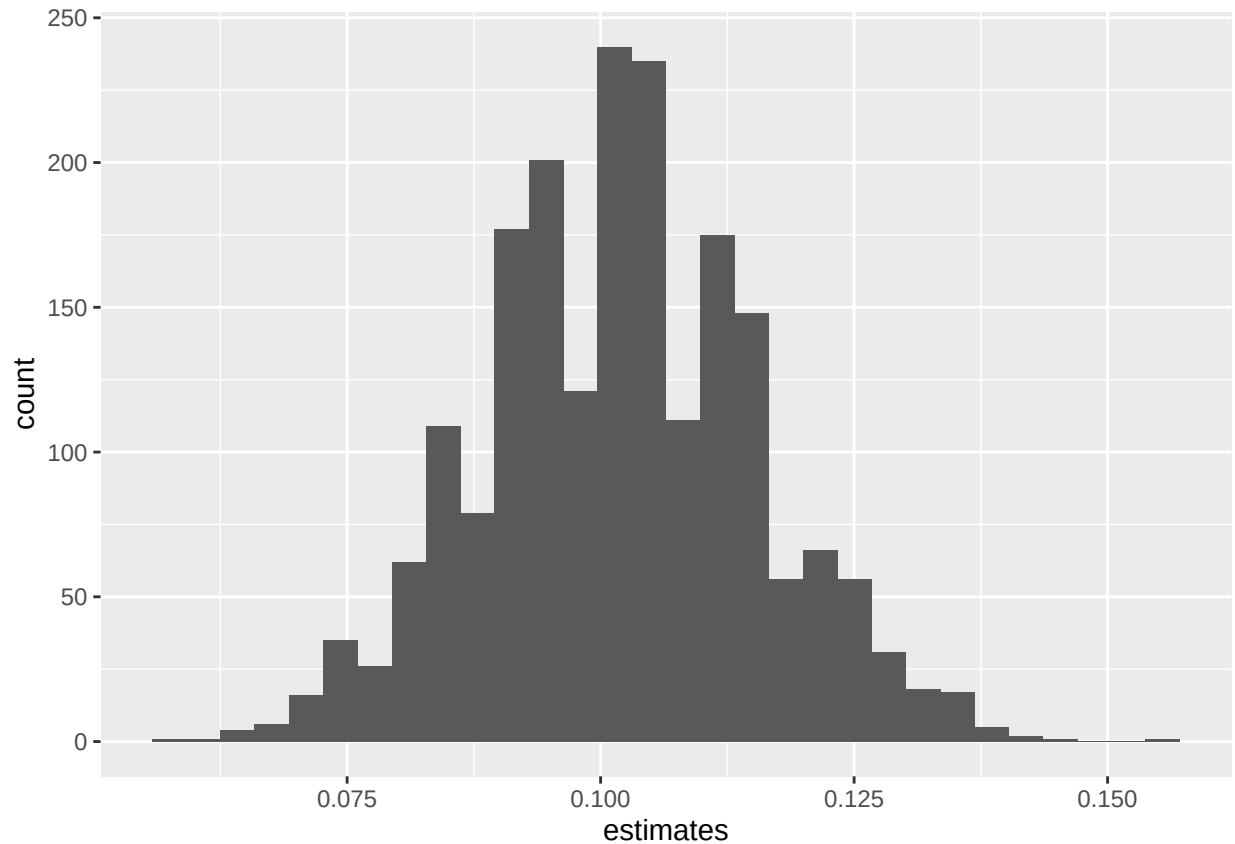
```
mean(birth_data$lbw)
```

```
## [1] 0.1023309
```

```
nrep = 2000
estimates_n_500 = rep(NA,nrep)
for (i in 1:nrep){
  set.seed(i)
  random_sample_n500 <- birth_data %>% slice_sample(n = 500)
  estimates_n_500[i] <- mean(random_sample_n500$lbw)
```

```
}
histogram_500 <- ggplot(data = data.frame(estimates = estimates_n_500), aes(x=estimates)) + geom_histogram()
histogram_500
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



```
mean(estimates_n_500)
```

```
## [1] 0.101866
```

```
sd(estimates_n_500)
```

```
## [1] 0.01347148
```

```
mean(estimates_n_500)
```

```
## [1] 0.101866
```

```
mean(birth_data$lbw)
```

```
## [1] 0.1023309
```

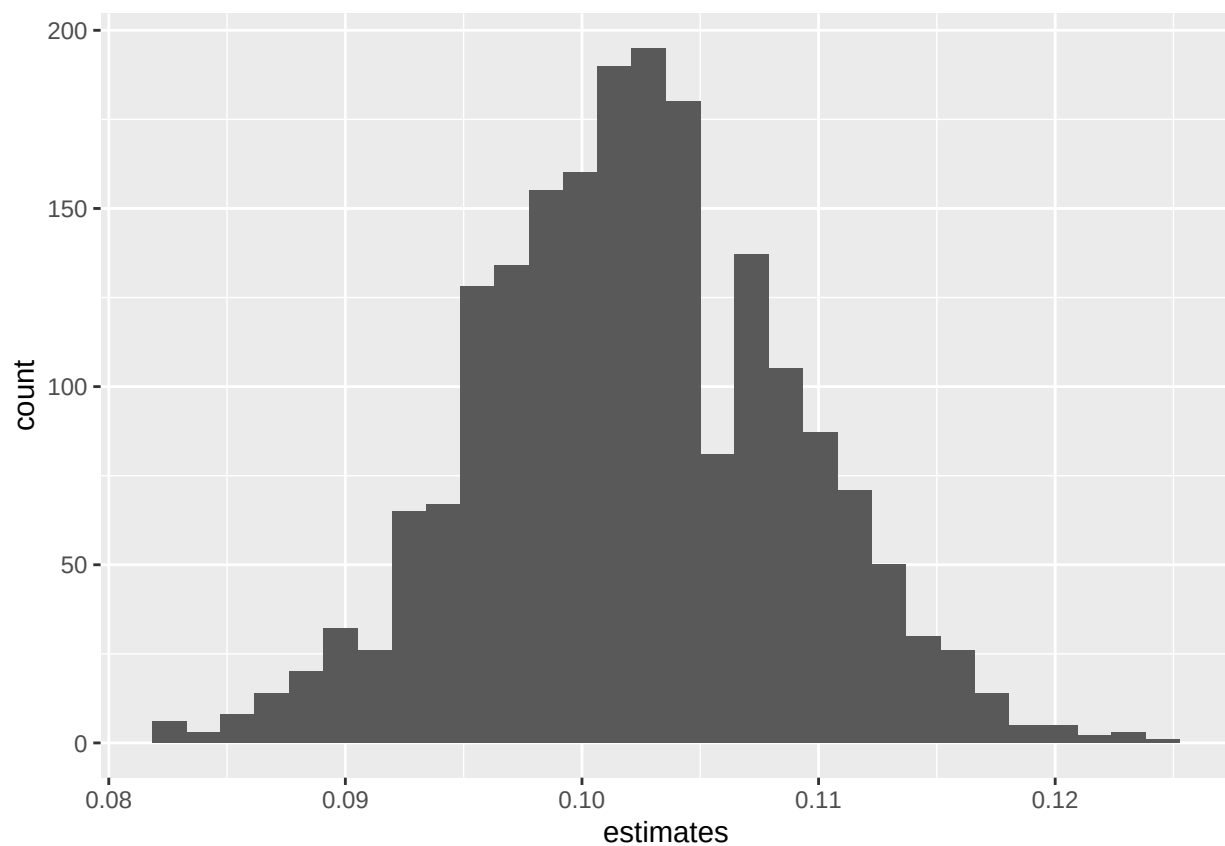
```

nrep = 2000
estimates_n_2000 = rep(NA,nrep)
for (i in 1:nrep){
  set.seed(i)
  random_sample_n2000 <- birth_data %>% slice_sample(n = 2000)
  estimates_n_2000[i] <- mean(random_sample_n2000$lbw)
}

histogram_2000 <- ggplot(data = data.frame(estimates = estimates_n_2000), aes(x=estimates)) + geom_histogram()
histogram_2000

```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



```
mean(estimates_n_2000)
```

```
## [1] 0.102211
```

```
sd(estimates_n_2000)
```

```
## [1] 0.006648878
```

```
mean(estimates_n_2000)
```

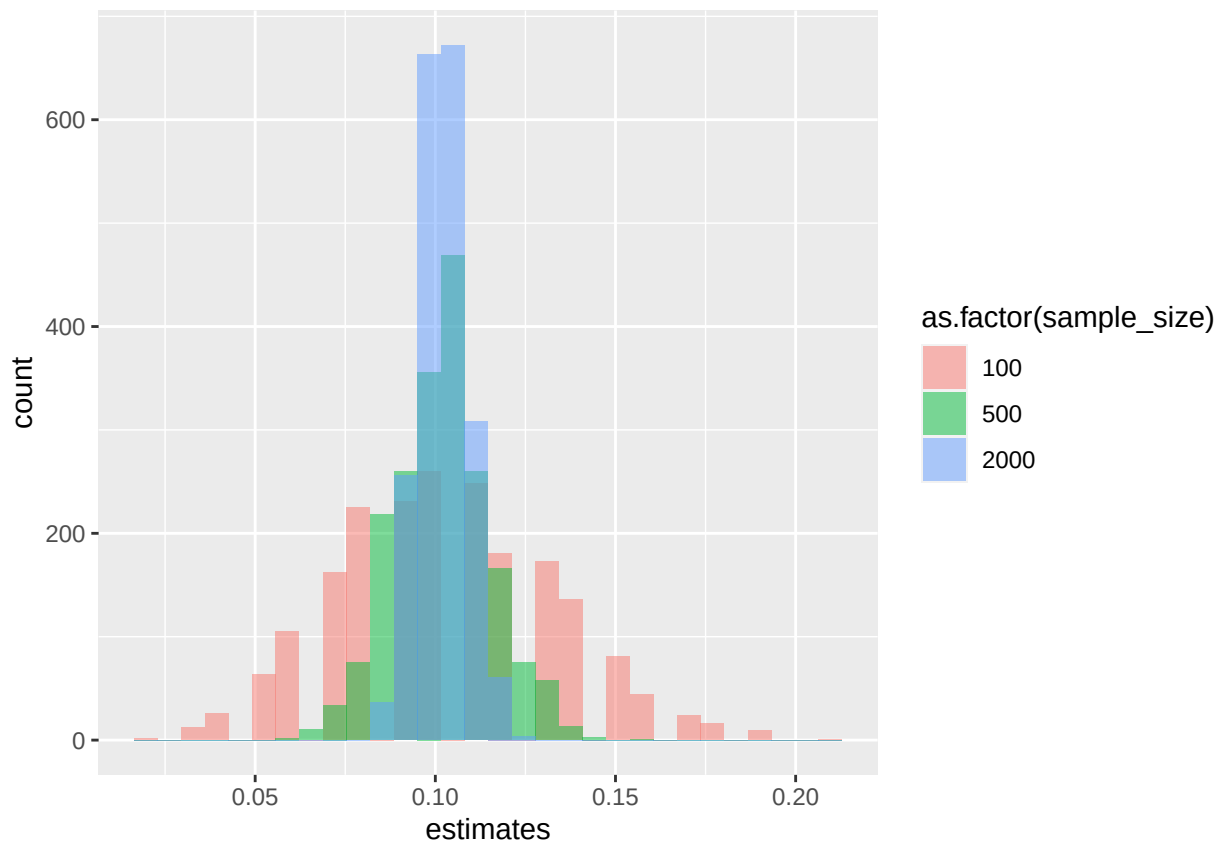
```
## [1] 0.102211
```

```
mean(birth_data$lbw)
```

```
## [1] 0.1023309
```

```
all_estimates = data.frame(estimates = c(estimates_n_100,estimates_n_500,estimates_n_2000), sample_size = c(100,500,2000))  
ggplot(data = all_estimates,aes(x=estimates, fill=as.factor(sample_size))) +  
  geom_histogram(alpha=0.5, position="identity")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



```
all_estimates %>% group_by(sample_size) %>% summarize(mean_estimate=mean(estimates), sd_estimate=sd(estimates))
```

```
## # A tibble: 3 x 3  
##   sample_size mean_estimate sd_estimate  
##   <dbl>         <dbl>         <dbl>  
## 1     100         0.103         0.0306  
## 2     500         0.102         0.0135  
## 3    2000         0.102         0.00665
```